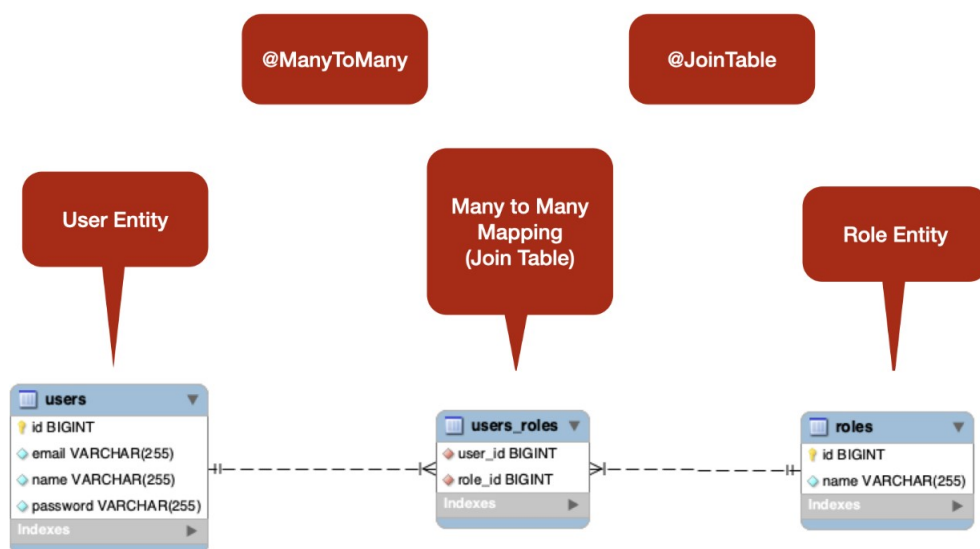


資料庫身份驗證的 Spring 安全登錄表單範例

在本 Spring 安全性教程中，我們將學習如何使用 Spring 安全性和 MySQL 資料庫實現具有資料庫身份驗證的自定義登錄表單。

在資料庫身份驗證的情況下，使用者在登錄表單中輸入使用者名和密碼等登錄憑據並提交表單。接下來，我們將使用資料庫表中的使用者名和密碼驗證使用者輸入的登錄。



資料庫設置

添加 Maven 依賴項

將以下 Maven 依賴項新增到您的 Spring 引導項目中：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

配置 MySQL 資料庫

讓我們首先使用以下命令在 MySQL 伺服器中建立資料庫：

```
create database loginDB
```

由於我們使用 MySQL 作為我們的資料庫，我們需要配置資料庫 **URL**，使用者名和密碼，以便 Spring 可以在啟動時與資料庫建立連接。打

開 `src/main/resources/application.properties` 檔，並向其添加以下屬性：

```
spring.datasource.url = jdbc:mysql://localhost:3306/loginDB
spring.datasource.username = root
spring.datasource.password = 1234

# Hibernate ddl auto (create, create-drop, validate, update)
spring.jpa.hibernate.ddl-auto = create-drop
```

```
logging.level.org.springframework.security=DEBUG
```

```
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.format_sql=true
```

```
#spring.jpa.hibernate.naming.implicit-strategy=org.hibernate.boot.model.nam  
ing.ImplicitNamingStrategyLegacyJpaImpl
```

```
#spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.nam  
ing.PhysicalNamingStrategyStandardImpl
```

模型層 - 建 JPA 實體

在此步驟中，我們將建使用者和角色 JPA 實體，並在它們之間建立多對多關係。

讓我們使用 JPA 註釋在使用者和角色實體之間建立多對多關係。

使用者

```
import jakarta.persistence.*;  
import lombok.AllArgsConstructor;  
import lombok.Getter;  
import lombok.NoArgsConstructor;  
import lombok.Setter;  
import java.util.Set;  
  
@Setter  
@Getter  
@NoArgsConstructor  
@AllArgsConstructor  
@Entity  
@Table(name = "users")  
public class User {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;
```

```

    private String name;
    @Column(nullable = false, unique = true)
    private String username;
    @Column(nullable = false, unique = true)
    private String email;
    @Column(nullable = false)
    private String password;
    @ManyToMany(fetch = FetchType.EAGER, cascade = CascadeType.ALL)
    @JoinTable(name = "users_roles",
        joinColumns = @JoinColumn(name = "user_id", referencedColumnName = "id"),
        inverseJoinColumns = @JoinColumn(name = "role_id", referencedColumnName = "id")
    )
    private Set<Role> roles;
}

```

角色

```

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "roles")
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
}

```

```
    private String name;
}
```

Repository

使用者存儲庫

```
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

    Boolean existsByEmail(String email);

    Optional<User> findByUsernameOrEmail(String username, String email);

    boolean existsByUsername(String username);
}
```

角色存儲庫

```
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Map;
import java.util.Optional;

public interface RoleRepository extends JpaRepository<Role, Long> {

    Optional<Role> findByName(String name);
}
```

服務層 - 自定義使用者詳細資訊服務

讓我們編寫一個邏輯，通過名稱或電子郵件從資料庫載入使用者詳細資訊。

讓我們創建一個 `CustomUserDetailsService`，它實現了 `UserDetailsService` 介面（Spring 安全內置介面），併為 `loadUserByUsername()` 方法提供了一個實現：

```
import lombok.AllArgsConstructor;
import net.javaguides.todo.entity.User;
import net.javaguides.todo.repository.UserRepository;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Set;
import java.util.stream.Collectors;

@Service
@AllArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String usernameOrEmail) throws
    UsernameNotFoundException {

        User user = userRepository.findByUsernameOrEmail(usernameOrEmail,
            usernameOrEmail)
            .orElseThrow(() -> new UsernameNotFoundException("User not
            exists by Username or Email"));

        Set<GrantedAuthority> authorities = user.getRoles().stream()
```

```
        .map((role) -> new SimpleGrantedAuthority(role.getName()))
        .collect(Collectors.toSet());

    return new org.springframework.security.core.userdetails.User(
        usernameOrEmail,
        user.getPassword(),
        authorities
    );
}
```

Spring Security 使用 `UserDetailsService` 介面，該介面包含 `loadUserByUsername (String username)` 方法來查找 給定使用者名的使用者詳細資訊。

`UserDetails` 介面表示經過身份驗證的用戶物件，Spring Security 提供了 `org.springframework.security.core.userdetails.User` 的開箱即用實現。

Spring 安全配置

讓我們創建一個類 `SpringSecurityConfig` 並向其添加以下配置：

```
import lombok.AllArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration
.AuthenticationConfiguration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.util.matcher.AntPathRequestMatcher;
```

```

@Configuration
@AllArgsConstructor
public class SpringSecurityConfig {

    private UserDetailsService userDetailsService;

    @Bean
    public static PasswordEncoder passwordEncoder(){
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
    {
        http.csrf().disable()
            .authorizeHttpRequests((authorize) ->
                authorize.anyRequest().authenticated()
            ).formLogin(
                form -> form
                    .loginPage("/login")
                    .loginProcessingUrl("/login")
                    .defaultSuccessUrl("/welcome")
                    .permitAll()
            ).logout(
                logout -> logout
                    .logoutRequestMatcher(new
AntPathRequestMatcher("/logout"))
                    .permitAll()
            );
        return http.build();
    }

    @Bean
    public AuthenticationManager
authenticationManager(AuthenticationConfiguration configuration) throws
Exception {
        return configuration.getAuthenticationManager();
    }
}

```



```
}
```

在 Spring 安全性設定中指定登錄頁面時，您負責呈現該頁面。

我們使用 Spring 安全性提供的 `BCryptPasswordEncoder` 類來加密密碼。

Thymeleaf 範本 - 自定義登錄頁面

以下 Thymeleaf 範本生成符合 `/login` 登錄頁面的 HTML 登錄表單。

登錄表格 - `src/main/resources/templates/login.html`

```
<!DOCTYPE html>
<html lang="en"
      xmlns:th="http://www.thymeleaf.org"
>
<head>
  <meta charset="UTF-8">
  <title>Login System</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.c
ss"
      rel="stylesheet"

integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTWfSpd3yD65VohhpuuC
OmLASjC"
      crossorigin="anonymous">
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" th:href="@{/index}">Spring Security Custom
Login Example</a>
    <button class="navbar-toggler" type="button"
data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent"
aria-controls="navbarSupportedContent" aria-expanded="false"
aria-label="Toggle navigation">
```

```
        <span class="navbar-toggler-icon"></span>
    </button>
</div>
</nav>
<br /><br />
<div class="container">
    <div class="row">
        <div class="col-md-6 offset-md-3">

            <div th:if="${param.error}">
                <div class="alert alert-danger">Invalid Email or Password</div>
            </div>
            <div th:if="${param.logout}">
                <div class="alert alert-success"> You have been logged
out.</div>
            </div>

            <div class="card">
                <div class="card-header">
                    <h2 class="text-center">Login Form</h2>
                </div>
                <div class="card-body">
                    <form
                        method="post"
                        role="form"
                        th:action="@{/login}"
                        class="form-horizontal"
                    >
                        <div class="form-group mb-3">
                            <label class="control-label"> Email</label>
                            <input
                                type="text"
                                id="username"
                                name="username"
                                class="form-control"
                                placeholder="Enter email address"
                            />
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>
```

```
<div class="form-group mb-3">
  <label class="control-label"> Password</label>
  <input
    type="password"
    id="password"
    name="password"
    class="form-control"
    placeholder="Enter password"
  />
</div>
<div class="form-group mb-3">
  <button type="submit" class="btn btn-primary"
>Submit</button>
</div>
</form>
</div>
</div>
</div>
</div>
</body>
</html>
```

自訂登錄 HTML 表單有幾個關鍵點：

- 表單應向 */login* 執行發佈。
- 表單應在名為 *username* 的參數中指定使用者名。
- 表單應在名為 *password* 的參數中指定密碼。
- 如果找到名為 *error* 的 HTTP 參數，則表示使用者未能提供有效的使用者名或密碼。
- 如果找到名為 *register* 的 HTTP 參數，則表示使用者已成功註銷。
- 許多使用者只需要自定義登錄頁面即可。但是，如果需要，您可以使用其他配置自定義前面顯示的所有內容。

Spring MVC 控制器

讓我們在 Spring MVC 控制器中建一個處理程式方法，它將 GET */Login* 映射到我們建的登錄範本：

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class WelComeController {

    @GetMapping("/welcome")
    public String greeting() {
        return "welcome";
    }

    @GetMapping("/login")
    public String login(){
        return "login";
    }
}
```

請注意，我們還創建了一個歡迎處理程式方法來返回**歡迎**百里香葉範本頁面。

ThymeLeaf 範本 - welcome.html

使用者登錄成功後，將顯示此歡迎頁面：

```
<!DOCTYPE html>
<html lang="en"
    xmlns:th="http://www.thymeleaf.org"
>
<head>
    <meta charset="UTF-8">
    <title>Registration and Login System</title>
```

```

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.c
ss"
    rel="stylesheet"

integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTWfSpd3yD65VohhpuuC
OmLASjC"
    crossorigin="anonymous">
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
        <a class="navbar-brand" th:href="@{/index}">Spring Security Custom
Login Example</a>
        <button class="navbar-toggler" type="button"
data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent"
aria-controls="navbarSupportedContent" aria-expanded="false"
aria-label="Toggle navigation">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarSupportedContent">
            <ul class="navbar-nav me-auto mb-2 mb-lg-0">
                <li class="nav-item">
                    <a class="nav-link active" aria-current="page"
th:href="@{/logout}">Logout</a>
                </li>
            </ul>
        </div>
    </div>
</nav>
<br /><br />

<body>
<div class="container">
    <div class="row">
        <h1> Welcome to Spring Security world!</h1>
    </div>
</div>

```

```
</body>
</html>
```

SQL Script

在測試 Spring 安全性之前，請確保使用以下 SQL 文稿將資料庫插入到相應的表中：

```
INSERT INTO `users` VALUES
(1,'mary@gmail.com','mary','$2a$12$X7NYFly1NN.3esamVsU870oUUfSu4iC0yx5ZPeXugd.3BH9Yy1NJy','mary'),
(2,'admin@gmail.com','admin','$2a$10$gqHrslMttQWSsDSVRTK10ehkkBiXsJ/a4z2OURU./dizw0Qu5Lovu','admin');

INSERT INTO `roles` VALUES (1,'ROLE_ADMIN'),(2,'ROLE_USER');

INSERT INTO `users_roles` VALUES (2,1),(1,2);
```

Hibernate 將自動創建資料庫表，因此您無需手動創建表。

使用瀏覽器使用資料庫身份驗證測試登錄表單

在瀏覽器中輸入 <http://localhost:8080> URL，它將導航到登錄頁面。

接下來，輸入使用者名作為 **admin**，密碼作為 **admin**，然後按下登錄

按鈕：

成功登錄后，您將看到以下網頁：接下來，按兩下應用程式中的註銷按鈕以從應用程式註銷：

結論

在本 Spring 安全性教程中，我們學習了如何使用 Spring 安全性和 MySQL 資料庫實現具有資料庫身份驗證的自定義登錄表單。